PROJECT TITLE:

Password Protected Door Lock System Using ATmega328P with Keypad and LCD Interface

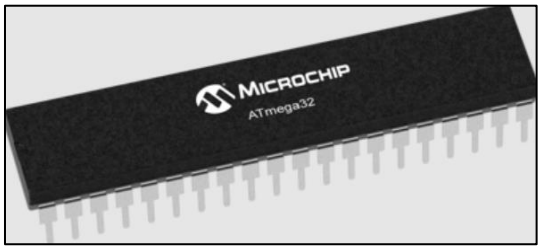
OBJECTIVE:

To design and implement a microcontroller-based password protection system interfacing a keypad as an analog input device, external memory for program/data storage, and an LCD display output device. This system restricts access through password authentication and provides feedback via a servo lock, LEDs.

COMPONENTS USED:

ATmega328P Microcontroller

The **ATmega328P** is an 8-bit AVR RISC-based microcontroller widely used in embedded systems, especially in Arduino platforms. It features 32KB of Flash memory, 2KB SRAM, and 1KB EEPROM, providing ample space for code and data storage. With 23 general-purpose I/O pins, 3 timers, 10-bit ADC (6 channels), USART, SPI, and I2C interfaces, it supports various peripherals and communication needs. In this project, the ATmega328P handles keypad input, compares it with the stored password, controls a servo motor for locking/unlocking, and updates the LCD and LED indicators. Its low power consumption, high reliability, and ease of programming make it an excellent choice for embedded security systems like password-protected door locks..



4x3 Keypad

The 4x3 matrix keypad consists of 12 keys arranged in 4 rows and 3 columns. It allows the user to input numeric values or special characters like '*' and '#'. When a key is pressed, the microcontroller detects the row-column connection and determines which key was pressed. This keypad is simple, inexpensive, and widely used in embedded systems. It serves as the primary input interface for entering passwords.

16x2 I2C LCD Display

The 16x2 LCD with I2C interface displays system messages such as password prompts, input digits, and access status. The I2C module reduces the number of control wires to just two (SDA and SCL), simplifying the wiring process. Each line of the display can hold up to 16 characters. It provides clear, real-time feedback to the user, improving the system's usability.

Servo Motor

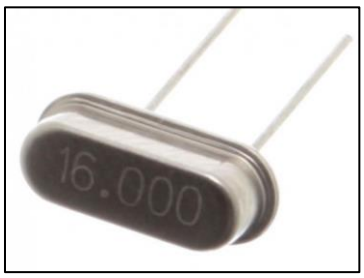
The servo motor acts as a physical locking mechanism. Controlled by PWM signals, it rotates to 90° when the correct password is entered, simulating an unlocked state, and returns to 0° after a short delay to lock the system again. It’s compact, energy-efficient, and easy to interface with microcontrollers. Its motion makes the access control system more tangible and interactive.

LEDs (Red and Green)

Two LEDs provide visual feedback: green for access granted, and red for access denied. They are connected via current-limiting resistors to prevent damage. These indicators allow the user to quickly understand the result of their password attempt. LEDs are essential for status indication in embedded systems due to their low power and high visibility.

16 MHz Crystal Oscillator

This crystal provides a stable and accurate clock source for the ATmega32, enabling consistent timing and reliable execution. It is critical for maintaining the proper frequency for internal operations like timing delays and communication protocols. Without a crystal oscillator, the microcontroller's internal clock may drift, causing instability.



22pF Capacitors

Used with the crystal oscillator, these capacitors stabilize oscillations and ensure that the microcontroller’s clock signal is clean and noise-free. They are typically connected between each leg of the crystal and ground. Even though small in size, they play a vital role in reliable system performance.

Power Supply (5V)

A regulated 5V power source is required to operate the ATmega32 and its peripherals. In this setup, an Arduino Uno board is used only for supplying 5V and GND—not for control or processing. Stable power is essential to prevent erratic behavior and component damage.

DESCRIPTION OF CIRCUIT:

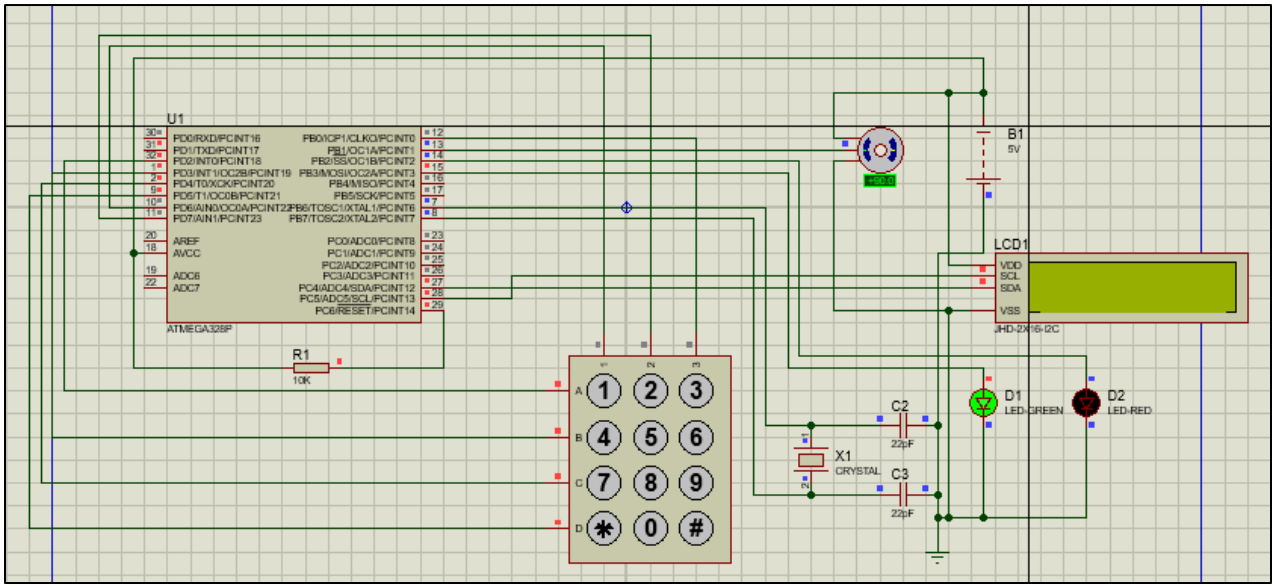
The circuit is designed around the ATmega32 microcontroller. It features:
- A keypad connected to PORTA and PORTB for password input.
- An I2C LCD for displaying status messages.
- A servo motor on Pin D6 to control a lock mechanism.
- Red and Green LEDs for access feedback.
- A buzzer to alert on access events.
- External crystal oscillator (16 MHz) for clocking the microcontroller.

FUNCTIONAL OVERVIEW:

This system is built around the **ATmega328P microcontroller**, with **Arduino Uno used solely to supply 5V power** not as a controller board.

1. Initial State:
   - LCD displays: "Enter Password:"
   - Servo is locked (0°). Both LEDs are off.
2. Password Entry via Keypad:
   - Users press numeric keys. Input is shown on LCD second line
3. Validation:
   - Pressing `#` checks the password.
   - If correct:
     - Green LED lights up.
     - Servo rotates to 90° to simulate unlock.
     - LCD shows "Access Granted".
     - Servo returns to 0° after 3 seconds.
   - If incorrect:
     - Red LED blinks.
     - LCD shows "Wrong Password".
4. Reset Input:
   - Pressing `*` clears the input and LCD.

SIMULATION:



SIMULATION OVERVIEW:

Simulated Components:

- Microcontroller (ATmega328P).
- 4x3 Matrix Keypad – Simulated as the user input device for entering the password.
- 16x2 I2C LCD Display – Used to display prompts, entered digits, and access status.
- Servo Motor – Represents a lock mechanism, rotating on successful authentication.
- Red and Green LEDs – Indicate failure or success of password entry.

Simulation Behavior:

1. The LCD displays "Enter Password:" at startup.
2. User inputs digits using the keypad.
3. The system validates the password upon pressing '#':
    - Correct Password:
        - Green LED turns ON.
        - LCD shows "Access Granted".
        - Servo rotates to unlock (90°), then returns to lock (0°).
    - Incorrect Password:
        - Red LED blinks.
        - LCD shows "Wrong Password".
4. Pressing '*' clears the input and resets the prompt.
5. All actions are simulated in real-time within the Proteus environment, closely reflecting hardware behavior.

CODE:

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <Keypad.h>

#include <Servo.h>


Servo myServo;

const int servoPin = 9;

LiquidCrystal_I2C lcd(0x27, 16, 2); // I2C address 0x27


const byte ROWS = 4;

const byte COLS = 3;

char keys[ROWS][COLS] = {

  {'1', '2', '3'},

  {'4', '5', '6'},

  {'7', '8', '9'},

  {'*', '0', '#'}

};

byte rowPins[ROWS] = {2, 3, 4, 5};

byte colPins[COLS] = {6, 7, 8};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);


const int redLED = 10;

const int greenLED = 11;


// Password settings
```

```
char password[] = "1234";
char inputBuffer[4];
int bufferIndex = 0;

void setup() {
  Serial.begin(9600);

  myServo.attach(servoPin);
  myServo.write(0);

  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Enter Password:");

  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  digitalWrite(redLED, LOW);
  digitalWrite(greenLED, LOW);
}

void loop() {
  char key = keypad.getKey();

  if (key) {
```

```
// Handle numeric keys (password input)
if (key >= '0' && key <= '9') {
  if (bufferIndex < 4) {
    inputBuffer[bufferIndex] = key;
    lcd.setCursor(bufferIndex, 1);
    lcd.print('*');  // Mask the input
    bufferIndex++;
  }

  // Check after 4 digits
  if (bufferIndex == 4) {
    bool correct = true;
    for (int i = 0; i < 4; i++) {
      if (inputBuffer[i] != password[i]) {
        correct = false;
        break;
      }
    }

    lcd.clear();
    lcd.setCursor(0, 0);

    if (correct) {
      lcd.print("Access Granted!");
      digitalWrite(greenLED, HIGH);
      digitalWrite(redLED, LOW);
```

```
      myServo.write(90);  // Open servo
    } else {
      lcd.print("Access Denied!");
      digitalWrite(greenLED, LOW);
      digitalWrite(redLED, HIGH);
      myServo.write(0);  // Lock servo
    }

    // Reset input buffer
    bufferIndex = 0;
    delay(2000);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Enter Password:");
  }
}

// Handle special keys
else {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Enter Password:");
  bufferIndex = 0; // Reset input buffer

  if (key == '*') {
    myServo.write(0);
```

```
      digitalWrite(redLED, LOW);

      digitalWrite(greenLED, LOW);

    }

    else if (key == '#') {

      digitalWrite(redLED, LOW);

      digitalWrite(greenLED, LOW);

    }

  }

 }

}
```

CODE FLOW EXPLANATION

1. System Initialization (setup()):
    - LCD is initialized and displays "Enter Password:”
    - Green and red LEDs are set as outputs and turned off.
    - The servo motor is attached and set to the locked position (0°).
2. Main Execution Loop (loop()):
    - Continuously listens for keypresses from the keypad.
3. Input Handling:
    - If a digit (0–9) is pressed:
        - It is added to the input string (up to 6 characters).
        - The input is displayed on the second line of the LCD.
    - If '*' is pressed (Reset):
        - Clears the input string.
        - Displays "Cleared" briefly, then resets the prompt.
    - If '#' is pressed (Submit):
        - Compares the entered password with the stored one ("1234").
        - If correct:
            - Displays "Access Granted".
            - Green LED turns on, servo unlocks (90°), then locks again after delay.
        - If incorrect:
            - Displays "Wrong Password".
            - Red LED turns on briefly as an alert.
            - Input is cleared and the system is ready for the next entry.

HARDWARE:

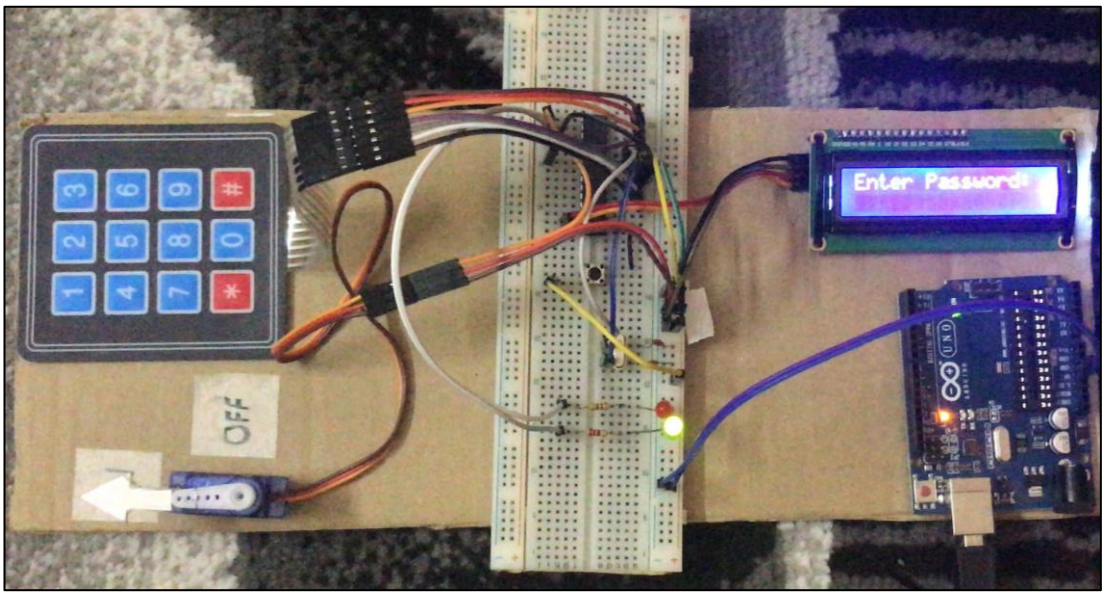


Figure 1: password is correct

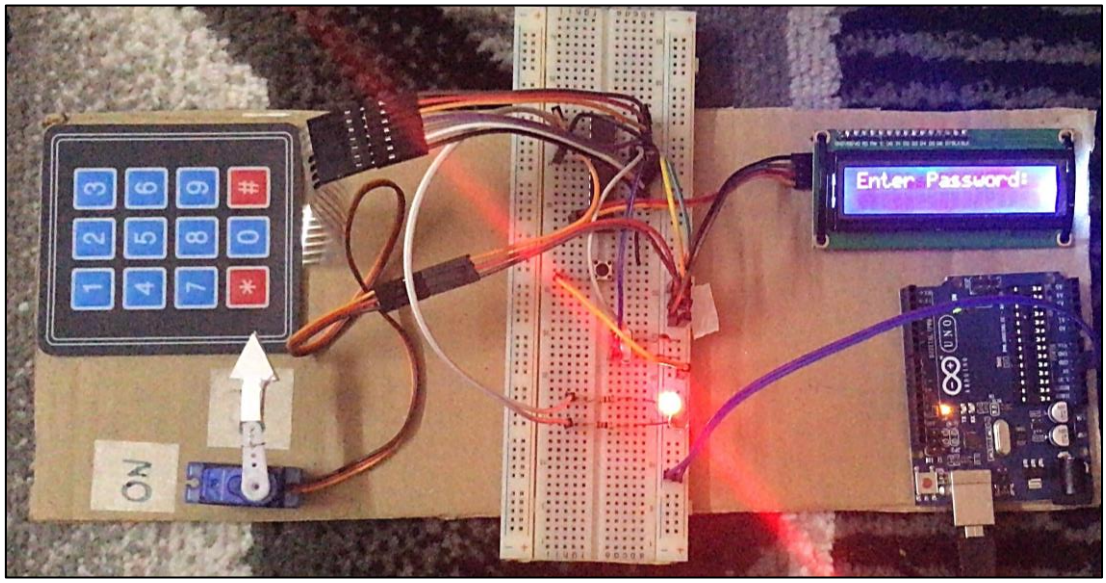


Figure 2: password is incorrect

CONCLUSION:

The project successfully demonstrates a secure, microcontroller-based password protection system using the ATmega328P. It integrates hardware components such as a keypad, servo motor, LEDs, and an LCD to create an interactive and responsive access control mechanism.